

❑ OLED Display Grid Extension

Software Overview

✖ What is it?

When building games like *Tic-Tac-Toe* or *Connect 4*, managing individual *x* and *y* pixels for every single game piece quickly becomes cumbersome.

The **Grid Extension** (`kcomp_oled_grid.h`) acts as a bridge between logic and the screen. It allows you to stop thinking in pixels and start thinking in *Rows* and *Columns*. It calculates the pixel offsets, draws borders, and manages memory, allowing you to focus on the rules of your game.

⚡ Memory Architecture (Bit-Crunching)

Because RAM on the ATmega328P is very limited, the Grid Extension does *bit-crunching*. Instead of dedicating a full byte (256 states) to a single cell, the library splits each byte into two 4-bit nibbles.

- **The Rule:** Two grid cells share a single byte of RAM.
- **The Constraint:** A grid cell can only hold a state between 0 and 15. (0 usually means empty).
- **The Memory Macro:** You must claim your memory globally at the top of your program using the macro: `GRID_MEM_SIZE(rows, cols)`.

📏 Coordinate System (The Anchor Point)

The grid does not hide the physical screen from you. When the grid asks you to draw a game piece, it gives you the **Anchor Point** (`anchorX`, `anchorY`) of that specific cell.

- The Anchor is always the absolute **Top-Left corner** of the cell on the physical OLED screen.
- You use the anchor to calculate where to draw (e.g., a circle at `anchorX + width/2`).

Software & Programming

📁 `kcomp_oled_grid.h`

To use the Grid Extension, you must take care of:

1. **The Display Library** The Grid Extension relies on the OLED library. You must include **exactly one** of following libraries at the top of your sketch:
 - `#include "kcomp_oled.h"`
 - `#include "kcomp_oled_paged.h"`
2. **The Grid Library**
Include the grid library directly afterwards: `#include <kcomp_oled_grid.h>`
 - **The Memory Declaration** The Grid Extension does not allocate memory itself. You must take ownership of the RAM required for your board by declaring a global array using the `GRID_MEM_SIZE(rows, cols)` calculator. (See the sample program!)

⚙️ Function Reference - Setup & Initialization

These functions configure the physical dimensions and memory of your grid.

Function Name	Description & Parameters
---------------	--------------------------

<code>void initGrid(uint8_t rows, uint8_t cols, int16_t cellW, int16_t cellH, int16_t offsetX, int16_t offsetY, uint8_t* memoryPointer)</code>	Setup: Binds your parameters and memory array to the engine. ▪ <code>rows, cols</code>: Grid dimensions. ▪ <code>cellW, cellH</code>: Width and height of a single cell in pixels. ▪ <code>offsetX, offsetY</code>: Starting absolute pixel of the entire grid (Top-Left). ▪ <code>memoryPointer</code>: The name of your <code>GRID_MEM_SIZE</code> array.
<code>void attachGridRender(GridRenderCallback callback)</code>	Hook: Connects your custom piece-drawing function to the engine. ▪ <code>callback</code>: The name of your custom function.
<code>void attachCursorRender(GridCursorCallback callback)</code>	Hook: Connects your custom UI cursor-drawing function to the engine. ▪ <code>callback</code>: The name of your custom function.



Function Reference - State Management

Use these to change the logical rules of your game. *Note: 0 is considered an empty cell.*

Function Name	Description & Parameters
<code>void clearGrid()</code>	Erase: Instantly wipes all cells back to state 0.
<code>void setGridCell(uint8_t row, uint8_t col, uint8_t state)</code>	Write: Changes the value of a specific cell. ▪ <code>row, col</code>: Target coordinates. ▪ <code>state</code>: An integer between 0 and 15.
<code>uint8_t getGridCell(uint8_t row, uint8_t col)</code>	Read: Returns the current state of a cell. ▪ Returns: 0 if empty, or 1 to 15.



Function Reference - The Spatial Cursor

An overlay that makes navigating the grid easy. It stops at the edges of your board so you cannot crash the system by moving out of bounds.

Function Name	Description & Parameters
<code>void enableCursor(bool enable)</code>	Visibility: Turns the cursor rendering on or off. ▪ <code>enable</code>: <code>true</code> to show, <code>false</code> to hide.
<code>void setCursorPos(uint8_t row, uint8_t col)</code>	Teleport: Instantly moves the cursor to a specific cell. ▪ <code>row, col</code>: The target coordinates.
<code>void moveCursor(int8_t deltaRow, int8_t deltaCol)</code>	Navigate: Moves the cursor relative to its current position (e.g., -1, 0 moves UP). Automatically constrained by grid edges. ▪ <code>deltaRow, deltaCol</code>: The amount to move by.
<code>uint8_t getCursorRow()</code>	Location: Returns the current row the cursor.
<code>uint8_t getCursorCol()</code>	Location: Returns the current column the cursor.



Function Reference - Hardware Rendering

Call these inside your display loop to push the grid to the physical screen.

Function Name	Description & Parameters
---------------	--------------------------

<code>void drawGridFramework()</code>	Borders Only: Draws the physical white dividing lines between cells.
<code>void renderGrid()</code>	Master Render: Draws the grid, evaluates all cells, calls your custom drawing function for any cell that is >0, and draws the cursor on top.

⚠ Pitfalls & Troubleshooting

- **The Missing Array:** If you forget to define your memory array at the top of your program (e.g., `uint8_t grid_data[GRID_MEM_SIZE(3,3)];`), or forget to pass it into `initGrid()`, the grid will silently fail to draw or store any pieces.
- **Drawing Outside the Anchor:** When writing your `drawGamePieces`-callback, remember that `anchorX` and `anchorY` represent the absolute screen position of the cell. If you write `drawCircle(10, 10, ...)`, it will always draw in the top-left of the physical screen, ignoring the cell's actual location. Always tether your math to the anchor!

📄 Sample Program

This minimal program initializes a 3x3 grid, places a single 'X' in the top corner, and enables a joystick-controlled cursor. It uses the paged library.

```
#include <kcomp_oled_paged.h>
#include <oled_grid.h>
#include <kcomp_joystick.h>

// 1. Claim the system memory for a 3x3 grid
uint8_t myGameMemory[GRID_MEM_SIZE(3, 3)] = {0};

// 2. Define how game pieces look (Triggered automatically by the engine)
void drawPieces(int16_t anchorX, int16_t anchorY, int16_t w, int16_t h,
uint8_t state) {
    if (state == 1) {
        // Draw an 'X' using the anchor points
        drawLine(anchorX, anchorY, anchorX + w, anchorY + h, OLED_COLOR_WHITE);
        drawLine(anchorX + w, anchorY, anchorX, anchorY + h, OLED_COLOR_WHITE);
    }
}

// 3. Define how the selection cursor looks
void drawMyCursor(int16_t anchorX, int16_t anchorY, int16_t w, int16_t h) {
    drawRect(anchorX, anchorY, w, h, OLED_COLOR_INVERT); // Blinking box
}

void setup() {
    initDisplay();
    initJoystick();

    // Bind the settings and memory to the engine
    initGrid(3, 3, 20, 20, 34, 2, myGameMemory);

    // Attach rendering functions
    attachGridRender(drawPieces);
    attachCursorRender(drawMyCursor);

    enableCursor(true);

    // Place a test piece in Row 0, Col 0
    setGridCell(0, 0, 1);
}
```

```
}  
  
void loop() {  
    // --- INPUT LAYER ---  
    int joyX = getJoystickXValue();  
    int joyY = getJoystickYValue();  
  
    // Basic un-debounced movement  
    if (joyX > 250) moveCursor(0, 1); // Right  
    if (joyX < -250) moveCursor(0, -1); // Left  
    if (joyY > 250) moveCursor(-1, 0); // Up  
    if (joyY < -250) moveCursor(1, 0); // Down  
  
    // --- RENDERING LAYER ---  
    firstPage();  
    do {  
        // Let the engine draw the borders, pieces, and cursor automatically  
        renderGrid();  
    } while (nextPage());  
}
```

💡 **Try it out:** You can find a full sample program **SimpleGridOLED** in the K-Comp examples folder!